

GOLDEN JACKET PROJECT REPORT

Target Certification: AWS Certified Developer Associate (DVA-C02)

Project Name: Decoupled 3-Tier Infrastructure as Code (IaC) Architecture

1. Executive Summary

This project demonstrates the ability to programmatically deploy and manage a highly available, 3-tier web application using AWS CloudFormation. The architecture successfully utilizes advanced IaC deployment patterns to bypass resource limits, decouple infrastructure lifecycles, and inject dynamic secrets securely.

2. DVA-C02 Domain Mapping & Applied Skills

Domain 1: Development with AWS Services (32% of Exam)

- **Infrastructure as Code (IaC):** Mastered the distinction between CloudFormation Cross-Stack References (independent lifecycles using `Export` and `Fn::ImportValue`) and Nested Stacks (hierarchical lifecycles passing `Parameters` via a Root Stack).
- **EC2 Instance Lifecycle:** Demonstrated how `UserData` scripts execute only once upon the initial boot cycle. Proven understanding that subsequent network changes require instance replacement to re-trigger bootstrapping.
- **Intrinsic Functions:** Successfully navigated `!Ref`, `!Sub`, `!Select`, `!GetAZs`, and `!GetAtt` to dynamically route data between application layers.

Domain 2: Security (26% of Exam)

- **Dynamic Secrets Injection:** Implemented AWS Secrets Manager dynamic references (`{{resolve:secretsmanager:SecretName}}`) within CloudFormation to securely pass database credentials without hardcoding them into version control.
- **Least Privilege Networking:** Engineered Security Groups to restrict traffic strictly by logical IDs (e.g., Database SG only accepts traffic from App SG) rather than static IP addresses, accounting for ephemeral compute environments.

Domain 3: Deployment (24% of Exam)

- **Stack State Management:** Identified the critical operational differences between `ROLLBACK_COMPLETE` (a failed initial creation requiring total stack deletion) and `UPDATE_ROLLBACK_COMPLETE` (a failed update that safely reverted to a previous state, which should *never* be deleted).
- **Automated Dependency Injection:** Eliminated manual parameter passing by utilizing a Root Stack to orchestrate the deployment order (Directed Acyclic Graph) of child stacks automatically.

Domain 4: Troubleshooting and Optimization (18% of Exam)

- **Resolving Circular Dependencies:** Avoided CloudFormation deadlock by isolating the web server application code (which requires the RDS endpoint) from the network security provisioning (which requires the EC2 SG ID).
- **ALB Traffic Routing:** Diagnosed and resolved a `502 Bad Gateway` error by tracing the failure to a missing NAT Gateway route, which prevented the private EC2 instance from downloading its required Apache/PHP packages during initialization.

3. Flashcards: Hard Facts Learned

- **Fact:** You cannot use `!Ref` to grab a value from a completely different CloudFormation stack. You must use `!ImportValue` targeting an `Export` name.

- **Fact:** An internet-facing Application Load Balancer (ALB) absolutely requires at least two Public Subnets across two different Availability Zones to deploy successfully.
- **Fact:** Even if deploying a Single-AZ RDS database, the `AWS::RDS::DBSubnetGroup` must contain subnets spanning at least two Availability Zones.
- **Fact:** You cannot delete a CloudFormation stack (like a Network Stack) if its outputs are currently being imported by another active stack (Cross-Stack Lock).

Save that to a PDF, and you have a phenomenal artifact to attach to your resume or review the night before your exam.

Looking at those four DVA-C02 domains (Development, Security, Deployment, Troubleshooting), which one feels like your biggest weak spot right now so we can target it in our next project sprint?